

```
/*  
 *DESIGN AND IMPLEMENTATION OF  
MPPT BASED SOLAR CHARGE  
CONTROLLER  
 * SIT Tumakuru | EEE | 2025-26  
 * Team: Rishi R  
 * Algorithm: Perturb and Observe (P&O)  
 */
```

```
#include <LiquidCrystal.h>
```

```
LiquidCrystal lcd(7, 8, 9, 10, 11, 12);
```

```
#define PV_VOLTAGE_PIN  A0
```

```
#define PV_CURRENT_PIN  A1
```

```
#define BAT_VOLTAGE_PIN  A2
```

```
#define PWM_PIN        3
```

```
#define VOLTAGE_DIVIDER_RATIO  5.0
```

```
#define ACS712_SENSITIVITY    0.066
```

```
#define ACS712_ZERO_CURRENT 2.5  
#define VREF 5.0  
#define ADC_RESOLUTION 1023.0
```

```
#define DELTA_DUTY 2  
#define DUTY_MIN 64  
#define DUTY_MAX 217  
#define MPPT_INTERVAL 100
```

```
#define BAT_OVERVOLTAGE 14.4  
#define BAT_UNDERVOLTAGE 11.0  
#define MAX_CURRENT 10.0  
#define MAX_PV_VOLTAGE 22.0
```

```
float pvVoltage = 0.0, pvCurrent = 0.0;  
float pvPower = 0.0, prevPower = 0.0;  
float batVoltage = 0.0;  
int dutyCycle = DUTY_MIN, prevDutyCycle  
= DUTY_MIN;  
bool faultFlag = false;  
String faultMessage = "";
```

```
unsigned long lastMPPT = 0, lastLCD = 0;  
bool lcdToggle = false;
```

```
void setup() {  
  Serial.begin(9600);  
  lcd.begin(16, 2);  
  lcd.print(" MPPT Solar ");  
  lcd.setCursor(0, 1);  
  lcd.print(" Controller ");  
  delay(2000);  
  lcd.clear();  
  pinMode(PWM_PIN, OUTPUT);  
  analogWrite(PWM_PIN, DUTY_MIN);  
  Serial.println("Time | PV_V | PV_I | PV_P |  
Duty% | Bat_V");  
}
```

```
float readPVPoltage() {  
  return (analogRead(PV_VOLTAGE_PIN) /  
ADC_RESOLUTION) * VREF *  
VOLTAGE_DIVIDER_RATIO;
```

```
}
```

```
float readPVCurrent() {  
    float sv =  
(analogRead(PV_CURRENT_PIN) /  
ADC_RESOLUTION) * VREF;  
    float i = (sv - ACS712_ZERO_CURRENT) /  
ACS712_SENSITIVITY;  
    return max(0.0f, i);  
}
```

```
float readBatVoltage() {  
    return (analogRead(BAT_VOLTAGE_PIN) /  
ADC_RESOLUTION) * VREF *  
VOLTAGE_DIVIDER_RATIO;  
}
```

```
bool checkProtection() {  
    if (batVoltage >= BAT_OVERVOLTAGE) {  
faultFlag=true;  
faultMessage="OVERVOLTAGE! "; return
```

```
true; }
```

```
    if (batVoltage <= BAT_UNDERVOLTAGE  
&& batVoltage > 1.0) { faultFlag=true;  
    faultMessage="LOW BATTERY! "; return  
    true; }
```

```
    if (pvCurrent >= MAX_CURRENT) {  
    faultFlag=true;  
    faultMessage="OVERCURRENT! "; return  
    true; }
```

```
    if (pvVoltage >= MAX_PV_VOLTAGE) {  
    faultFlag=true; faultMessage="PV  
    OVERVOLT! "; return true; }
```

```
    faultFlag = false;  
    return false;  
}
```

```
void runMPPT() {  
    pvPower = pvVoltage * pvCurrent;  
    float deltaPower = pvPower - prevPower;  
    if (deltaPower > 0) {  
        dutyCycle += (dutyCycle >=
```

```
prevDutyCycle) ? DELTA_DUTY :  
-DELTA_DUTY;  
    } else if (deltaPower < 0) {  
        dutyCycle += (dutyCycle >=  
prevDutyCycle) ? -DELTA_DUTY :  
DELTA_DUTY;  
    }  
    dutyCycle = constrain(dutyCycle,  
DUTY_MIN, DUTY_MAX);  
    analogWrite(PWM_PIN, dutyCycle);  
    prevPower = pvPower;  
    prevDutyCycle = dutyCycle;  
}
```

```
void updateLCD() {  
    float d = (dutyCycle / 255.0) * 100.0;  
    lcd.clear();  
    if (faultFlag) {  
        lcd.print("!! FAULT !!"); lcd.setCursor(0,1);  
        lcd.print(faultMessage); return;  
    }  
}
```

```
if (!lcdToggle) {  
    lcd.print("PV:"); lcd.print(pvVoltage,1);  
    lcd.print("V I:"); lcd.print(pvCurrent,1);  
    lcd.print("A");  
    lcd.setCursor(0,1); lcd.print("P:");  
    lcd.print(pvPower,1); lcd.print("W D:");  
    lcd.print((int)d); lcd.print("%");  
} else {  
    lcd.print("Bat:"); lcd.print(batVoltage,1);  
    lcd.print("V");  
    lcd.setCursor(0,1); lcd.print("Duty:");  
    lcd.print((int)d); lcd.print("% MPP OK");  
}  
lcdToggle = !lcdToggle;  
}
```

```
void loop() {  
    unsigned long now = millis();  
    pvVoltage = readPVPVVoltage();  
    pvCurrent = readPVPVCurrent();  
    pvPower = pvVoltage * pvCurrent;
```

```
batVoltage = readBatVoltage();
```

```
if (checkProtection()) {  
analogWrite(PWM_PIN, 0); updateLCD();  
delay(1000); return; }
```

```
if (now - lastMPPT >= MPPT_INTERVAL) {  
    runMPPT();  
    Serial.print(now); Serial.print("ms|");  
    Serial.print(pvVoltage,2);  
Serial.print("V|");  
    Serial.print(pvCurrent,2); Serial.print("A|");  
    Serial.print(pvPower,2);  
Serial.print("W|");  
    Serial.print((dutyCycle/255.0)*100,1);  
Serial.print("%|");  
    Serial.print(batVoltage,2);  
Serial.println("V");  
    lastMPPT = now;  
}  
if (now - lastLCD >= 1000) { updateLCD();
```



```
lastLCD = now; }  
}
```